

PHENIKAA UNIVERSITY
PHENIKAA SCHOOL OF COMPUTING



COURSE: SOFTWARE ARCHITECTURE

TOPIC:

QR CODE-BASED IN-TABLE FOOD ORDERING SYSTEM

FINAL REPORT

Code Course : CSE703110-1-3-25
Class : N01
Instructor : THS. Vũ Quang Dũng
Group : 01
Members : 1. Nguyễn Đình Trung - 23010701
2. Vũ Anh Nam - 23010886
3. Đặng Minh Hoàng – 23010445
4. Nguyễn Phan Quang Minh - 23010530

Student Name	Position	Task
Nguyễn Đình Trung	Leader	(1) Requirement Analysis (2) Architectural & Interface Design (3) Prepare Reports
Vũ Anh Nam	Member	(1) Requirement Analysis (2) Prepare Reports
Đặng Minh Hoàng	Member	
Nguyễn Phan Quang Minh	Member	(1) Prepare Reports (2) Testing Interface

Table of Contents

1. Executive Summary	4
2. Project Requirements & Goals	4
2.1. Core Functional Requirements.....	4
2.2. Key Quality Attributes.....	12
2.3. Propose model of ASR	13
2.4. Use Case Modeling	15
3. Architectural Design & Implementation	16
3.1. Architectural Pattern.....	16
3.2. System Context Diagram (C4 Level).....	20
3.3. Technical Stack and Data Model.....	25
3.4. Implementation Overview	27
4. Testing & Verification.....	29
4.1. Environment & Configuration	29
4.2. Functional Testing	30
4.3. Non-Functional Testing	31
4.4. API Testing.....	32
4.5. Validation Summary.....	33
5. Conclusion & Reflection.....	35
5.1. Lessons Learned.....	35
5.2. Future Improvements	36

1. Executive Summary

This project details the UI/UX design and prototyping of the QR Code-Based In-Table Food Ordering System, a digital e-menu platform developed to streamline tableside ordering and customer-to-staff interactions. The system addresses the critical challenges of delayed order taking and fragmented communication within busy food and beverage (F&B) environments.

From a design perspective, the project prioritizes intuitive user navigation and seamless micro-interactions over complex, multi-step workflows. To satisfy requirements for both visual scalability and interface flexibility, the system employs a centralized Design System paired with a Component-Driven Architecture in Figma. By integrating advanced interaction principles—such as interactive component variants for real-time quantity adjustments, synchronized radio button states for payment selection, and smart overlay mapping for instantaneous "Call Staff" modals—the architecture ensures high scannability while avoiding the cognitive overhead of traditional service models.

The system is built as a responsive, high-fidelity interactive prototype to guarantee cross-device consistency and simplify user journey validation. The interface’s robustness was validated through rigorous usability testing, while user performance benchmarks confirmed its reliability: test participants successfully completed a full checkout cycle within 45 seconds, maintaining a 100% task completion rate without external guidance. These results demonstrate that the chosen UI/UX framework effectively balances visual simplicity with high operational efficiency, providing a sustainable foundation for future full-stack development and POS system integration.

2. Project Requirements & Goals

2.1. Core Functional Requirements

a. Actors

Actor	Role / Responsibilities	Interaction With System
Customer	Acts as the primary end-user visiting the restaurant. Responsible for browsing the menu, selecting food items, managing choices, and completing payments	Scans table QR code to view the menu. Searches, filters, and adds items to cart. Places orders directly to the kitchen. Performs online payment or requests cash payment.

Restaurant Staff	Responsible for receiving order details and preparing dishes accurately. Ensures food is delivered to the correct table and assists customers.	Logs into the staff app to receive real-time orders. Updates food preparation status (Preparing/Ready). Manages table service and availability.
Administrator	Responsible for managing the digital menu, restaurant configuration, and staff accounts. Monitors daily business performance and financial revenue.	Accesses the central administrative dashboard. Manages digital menu (items, prices, stock). Generates table QR codes and layout. Views and exports sales/revenue reports
Payment Gateway	Responsible for verifying accounts and ensuring safe, encrypted monetary transactions.	Receives payment requests and invoice details. Processes the transaction from customer accounts. Returns success/failed status to the system.

b. Functional Requirements (FRs)

ID	Functional Requirement	Priority
FR-01	Automatically recognizes the table and displays the digital menu instantly without registration.	High
FR-02	Allows customers to search items by name and filter by categories (mains, drinks,...).	Medium
FR-03	Allows customers to add, modify quantity, delete items, and add special kitchen notes.	High
FR-04	Sends the cart items directly to the kitchen display and generates a unique Order ID.	High
FR-05	Supports online payments (MoMo, VNPay) or cash payment requests to alert servers.	High

c. Non-Functional Requirements (NFRs)

ID	Attribute	Description	Impact
NFR-01	Performance (Latency)	The digital menu and food images must fully load within less than 2 seconds after the QR scan. The shopping cart update latency must be under 500ms.	High
NFR-02	Security (Integrity)	All order data, payment information, and transaction statuses must be protected against unauthorized alteration or data leakage during transmission.	Critical
NFR-03	Reliability (Availability)	The platform must guarantee a 99.9% operational uptime during active restaurant business hours (7:00 AM - 11:00 PM).	Critical
NFR-04	Usability	The web interface must be intuitive, minimal, and fully responsive across various mobile screen sizes without requiring app installation.	Medium

d. Use Case List by Functional Group

UC1: Customer In-Table Ordering Management

UC-1.1 Browse e-Menu & Manage Cart	
Description	Customer scans the table's QR code to view categories, active items, customize options, and manage their local shopping cart.
Trigger	Customer opens the browser via table QR code or digital menu URL.
Preconditions	The system, item catalog, and database are available.
Postconditions	Selected items and modifications are successfully saved to the active local cart instance.
Normal Flows	1) Customer accesses the digital menu interface. 2) System displays active menu categories (e.g., Coffee, Smoothie) and available items. 3) Customer searches or filters items. 4) Customer adjusts item quantities, inputs special modifications, and adds them to the cart.
Alternative Flows	A1) Customer uses category tabs to jump directly to specific items. A2) Customer clicks the "Edit" link within the cart to return to the item details view for modification.
Exceptions	E1) Invalid or unmapped QR code scanned -> system displays error view. E2) System/DB connection failure.
Priority	High
Frequency of Use	High
Business Rules	Inactive or out-of-stock items must be hidden or greyed out; quantities cannot be negative.
Assumptions	Menu details include item code, name, price, category_id, availability status, and metadata.

UC-1.2 Place In-Table Order	
Description	Customer reviews the finalized cart items, selects a payment method, and submits the order to the kitchen.
Trigger	Customer clicks the "Đặt đơn" (Place Order) button within the shopping cart view.
Preconditions	Shopping cart must contain at least one valid item; Table ID must be authenticated.
Postconditions	Order data is locked and stored in the database; customer cart is cleared; real-time order alert is pushed to the staff dashboard.

Normal Flows	1) Customer opens the checkout/shopping cart view. 2) System displays items, itemized costs, total price, and payment selections. 3) Customer selects a payment method (Cash or Bank Transfer) and submits the order. 4) System validates current stock, processes submission, and returns an "Order Placed Successfully" temporary toast.
Alternative Flows	A1) Customer modifies item counts or removes rows prior to confirmation. A2) Customer cancels checkout and goes back to the homepage.
Exceptions	E1) Item stock drops to zero during checkout -> prompt user to adjust cart. E2) Order creation transaction fails -> show connection error toast.
Priority	High
Frequency of Use	High
Business Rules	Each submission must explicitly bind to a verified Table ID; payment method field cannot be blank.
Assumptions	Order metadata includes order_id, table_id, timestamp, total_amount, payment_status, and order_status.

UC-1.3 Query Tableside Assistance / History

Description	Customer triggers an assistance alert or checks their current table's order queue state.
Trigger	Customer clicks "Gọi nhân viên" (Call Staff) or "Món đã gọi" (Order History) on the header navigation bar.
Preconditions	Customer session with a valid Table ID is active.
Postconditions	An interactive overlay modal or history list view is rendered based on user action.
Normal Flows	1) Customer taps the assistance or history action trigger. 2) System checks active database states. 3) System presents either a list of help options or previously placed item cards.
Alternative Flows	A1) Customer clicks outside the modal bounding box to dismiss the overlay interaction.
Exceptions	E1) Connection timeout.
Priority	Medium
Frequency of Use	High
Business Rules	Order history view only pulls records that match the current table session context..

UC2: Staff Workflow & Resolution Management

UC-2.1 Process Incoming Order Status	
Description	Staff reviews newly broadcasted pending orders, updates progress, and dispatches them to the kitchen.
Trigger	Staff opens the Merchant Dashboard -> New Orders tab.
Preconditions	Staff is authenticated and authorized; real-time network layer is connected.
Postconditions	Order tracking record state shifts; active workspace logs the processed action.
Normal Flows	1) Staff opens order management. 2) System highlights incoming orders with red indicators. 3) Staff reviews details and clicks “Accept” & “Notify Kitchen” 4) System updates status to "In Progress" and transfers the card to the preparation tab.
Alternative Flows	A1) Staff rejects the order by pressing “Cancel” if criteria are unfulfilled.
Exceptions	E1) Concurrency collision: order already approved by another worker -> refresh layout. E2) Database error.
Priority	High
Frequency of Use	Medium
Business Rules	Only authenticated merchant accounts can alter system-wide order status states.
Assumptions	Active logging tracks staff_id, event_type, target_id, and time tokens.

UC-2.2 Resolve Customer Assistance Alerts	
Description	Staff views and dismisses critical tableside notifications triggered by seated customers.
Trigger	Critical pop-up warning "THÔNG BÁO KHẨN" overlays the staff dashboard view.
Preconditions	An active assistance request flag is logged inside the active message table.
Postconditions	Alert modal disappears; table status state resets; staff addresses the customer's request.
Normal Flows	1) System detects a live request and prompts an overlay modal detailing Table ID and target reason (e.g., [TÍNH TIỀN]). 2) Staff acknowledges text contents. 3) Staff hits the confirmation switch "ĐANG ĐẾN XỬ LÝ " (On My Way). 4) System clears the notification flag and returns the app

Alternative Flows	None.
Exceptions	E1) Data synchronization delay -> alert fails to close -> trigger automatic list polling.
Priority	High
Frequency of Use	Medium
Business Rules	Urgent assistance alerts possess absolute layout rendering hierarchy and block alternative actions until closed.
Assumptions	Merchant terminal app is continually running in an active foreground mode.

UC3: Menu & Catalog Management

UC-3.1 Manage Menu Items	
Description	Admin/Manager performs Add/Edit/Delete (or Deactivate)/Search/Display operations for food and beverage items in the digital catalog.
Trigger	Admin opens Menu Management -> Items.
Preconditions	Admin is authenticated and authorized; system and database are available.
Postconditions	Menu item data is updated according to the selected action; the customer-facing e-Menu list is instantly refreshed; audit logs are recorded.
Normal Flows	1) Admin accesses Menu Item management screen. 2) System displays the current list of items and available management actions. 3) Admin performs a management action 4) System validates data formats and pricing constraints, then applies and saves changes.
Alternative Flows	A1) Admin uses search keyword or filters by category (e.g., Coffee, Smoothie) to narrow down items. A2) Admin cancels an in-progress add/edit item form.
Exceptions	E1) Duplicate item code or item name. E2) Negative or invalid price format -> system prompts correction. E3) System/DB error.
Priority	High
Frequency of Use	High
Business Rules	Item code must be unique; required fields (Name, Price, Category) must be provided; pricing cannot be less than zero.
Assumptions	Item details include at least item code, name, price, image url, availability status (In stock/Out of stock), and updated metadata.
UC-3.2 Manage Menu Categories	

Description	Admin/Manager performs Add/Edit/Delete/Assign/Search/Display operations for menu categories (e.g., Coffee, Smoothie, Fruit Juice) to structure the e-Menu.
Trigger	Admin opens Menu Management -> Categories.
Preconditions	Admin is authenticated and authorized.
Postconditions	Category database structure is updated; customer interface updates the filter tabs accordingly.
Normal Flows	1) Admin opens category management screen. 2) Admin performs a management action (creates a new category, deactivates an old one, or reorders the list). 3) Admin saves changes.
Alternative Flows	A1) Admin creates a new category based on an existing template or duplicates an operational group structure.
Exceptions	E1) Duplicate category identifier/name. E2) Deletion blocked if the category currently contains active items linked to it -> system suggests deactivation instead.
Priority	Medium
Frequency of Use	Medium
Business Rules	Category identifier must be unique; each category should map to valid active menu structures.
Assumptions	Category data includes category code, name, display order, status, and metadata.

UC-3.3 Manage Tables & QR Codes	
Description	Admin performs Add/Edit/Delete/Assign/Search/Display operations for physical table records and generates distinct QR codes mapped to each table.
Trigger	Admin opens System Management -> Tables & QR Codes.
Preconditions	Admin is authenticated and authorized.
Postconditions	Table data is updated; target QR codes are uniquely generated, refreshed, or mapping files are downloaded; audit logs are recorded.
Normal Flows	1) Open table management screen. 2) Perform a management action (e.g., create a new table identifier "BÀN 06"). 3) System automatically generates a unique tokenized URL and corresponding QR code graphic for the table. 4) Save changes and download the QR vector/image file.
Alternative Flows	A1) Configure sorting order, floor/area filters (e.g., Ground Floor, Rooftop), or batch-generate QR codes for multiple tables.
Exceptions	E1) Duplicate table number/identifier code. E2) Database/QR generation API timeout.
Priority	High

Frequency of Use	Medium
Business Rules	Table code must be unique within the restaurant layout; only active tables can receive digital order data via the customer interface.
Assumptions	Table data includes table_id, table_number, zone_location, current_occupancy_status, and QR_code_url metadata.

UC-3.4 Manage Combo / Menu Structure	
Description	Admin performs Add/Edit/Delete/Assign/Search/Display operations for complex menu structures, organized by promotional Combos or custom recommendation groups.
Trigger	Admin opens Menu Management -> Combo & Menu Structure.
Preconditions	Admin is authenticated; valid individual food/beverage items exist in the database (from UC-3.1).
Postconditions	Custom structures, discount mappings, and recommendation metrics are updated and stored inside the active catalog.
Normal Flows	1) Select target area, time slot, or promotion version. 2) Open structure/combo editor screen. 3) Arrange categories and conditional groups (e.g., Choose 1 Coffee + 1 Pastry). 4) Assign specific menu items into the combo container. 5) Save changes.
Alternative Flows	A1) Reorder menu display hierarchies or modify group priority values.
Exceptions	E1) Constraint violations (e.g., setting a combo price higher than the sum of individual items).
Priority	High
Frequency of Use	High during seasonal menu/promotion updates.
Business Rules	Menu structures must comply with catalog structural rules; only active and in-stock menu items can be assigned into visible combos.
Assumptions	The system supports multi-item grouping and dynamic price calculation overrides for combo schemas.

UC-3.5 Manage Store Configuration & Tax Framework	
Description	Admin performs Add/Edit/Delete/Assign/Search/Display operations for global store rules, service charge constraints, and tax framework standards.
Trigger	Admin opens System Settings -> Configuration & Tax Framework.
Preconditions	Admin is authenticated and authorized.
Postconditions	Operational standards, VAT rules, and global store service fees are updated and versioned.

Normal Flows	1) Open framework management. 2) Configure standard system parameters (e.g., set VAT to 10%, apply 5% weekend service fee). 3) Save.
Alternative Flows	A1) Compare framework operational rules across different seasonal setups (if supported).
Exceptions	E1) Conflicts with active, processing table bills or pre-calculated order values.
Priority	High
Frequency of Use	Medium
Business Rules	Framework adjustments must not invalidate historical transaction summaries or completed order invoices; data versioning is recommended.
Assumptions	Framework structure includes tax percentages, extra fee rules, currency codes, and applicability time ranges.

UC-3.6 Copy Menu Catalog	
Description	Admin copies an entire existing menu schema/catalog to quickly create a new version for a different seasonal event, shift, or holiday theme.
Trigger	Admin selects "Copy Menu" on a target source catalog structure.
Preconditions	Source menu data exists; Admin possesses proper catalog duplicative permissions.
Postconditions	A new cloned menu version is generated for the target event/season container.
Normal Flows	1) Select source menu schema. 2) Choose target seasonal shift, holiday code, or active timeline. 3) Confirm transfer scope. 4) System duplicates data models. 5) Admin reviews the cloned catalog.
Alternative Flows	A1) Copy the complete menu layout architecture without duplication of base pricing lists if store pricing strategies differ.
Exceptions	E1) Target timeline or event container already holds an active menu version -> system prompts user for conflict resolution.
Priority	Medium
Frequency of Use	Medium
Business Rules	Copy must preserve data integrity; edits on the new version must not affect the source.
Assumptions	Menu catalog supports versioning by shifts, seasons, or promotional events.

UC-3.7 Display/Query Menu Catalog	
Description	Admin/Staff/Customer views the menu catalog structure based on category, event session, or menu version.
Trigger	Admin/Staff/Customer opens the main e-Menu or Catalog view interface.
Preconditions	Menu catalog data exists; viewing access is allowed.
Postconditions	The selected menu layout and item details are displayed correctly on the target device.
Normal Flows	1) User selects category/session/version. 2) System loads menu structure, categories, and items. 3) Display details (names, descriptions, pricing, image elements).
Alternative Flows	A1) Export Menu data schema to PDF/Excel file (if enabled for Admin/Staff).
Exceptions	E1) Missing or invalid selected menu version -> system falls back to default active catalog.
Priority	Medium
Frequency of Use	High
Business Rules	Customer sees only approved/active versions and available items; hidden elements are restricted to administrative views.
Assumptions	Filters by category/session/availability status are available.

UC4: Order Bill & Revenue Calculation

UC-4.1 Set Up Menu Item Pricing & Tax Unit	
Description	Admin configures the base price, active discount rules, and tax calculation basis (VAT, Service Charges) for billing formulas according to store regulations.
Trigger	Admin opens Finance -> Pricing & Fee Setup.
Preconditions	Admin is authenticated and authorized.
Postconditions	Base item prices, fee units, and tax calculation formulas are stored; audit logs are recorded.
Normal Flows	1) Open settings screen. 2) Input unit price updates, tax rules, or special surcharge percentages. 3) Validate mathematical ranges. 4) Save settings.
Alternative Flows	A1) Configure pricing modifiers based on specific table categories or VIP membership statuses if supported..
Exceptions	E1) Invalid input ranges (e.g., negative tax values) or business rule conflicts.
Priority	Medium
Frequency of Use	Medium
Business Rules	Tax frameworks must comply with legal standards; active modification of base pricing must instantly apply to newly created orders without changing already locked/completed

Assumptions	Core system calculations utilize standard double-entry precision formats to avoid currency rounding errors. .
--------------------	--

UC-4.2 Display Bill & Calculation	
Description	Admin/Staff/Customer views bill calculation information specific to a Table ID, active order session, or completed transaction.
Trigger	Admin/Staff/Customer opens View Bill/Checkout for a selected Table/Order.
Preconditions	Pricing units/tax rules are configured; view access is allowed.
Postconditions	Bill calculation information is displayed accurately.
Normal Flows	1) Select table/order session. 2) System retrieves/calculates total bill amount. 3) Display breakdown (subtotal, applied VAT/surcharges, and active discounts).
Alternative Flows	A1) Export bill/receipt information (if enabled for Admin/Staff accounts).
Exceptions	E1) Missing menu pricing configuration or failed item mapping for selection.
Priority	Medium
Frequency of Use	Medium
Business Rules	Displayed bill must align with the active, approved tax and item catalog configuration.
Assumptions	Calculation uses item quantities and configured base unit prices.

2.2. Key Quality Attributes

Architecturally Significant Requirements (ASRs)

ID	Quality Attribute	Description	Impact
ASR-01	Modifiability	Easily switch or add new payment gateways (e.g., MoMo, VNPay) without modifying the core ordering logic.	Isolate the payment feature into a separate, decoupled module to prevent code regression in other functions.
ASR-02	Security	Prevent client-side price manipulation and secure transaction data using standard HTTPS.	Enforce Server-side validation for all financial and cart calculations; never trust raw data sent from the browser.
ASR-03	Scalability	Handle concurrent order requests from multiple tables during peak hours without system freezing.	Keep application services stateless and optimize database queries to prevent processing bottlenecks.

2.3. Propose model of ASR

Propose Model of Architecturally Significant Requirements (ASR)

Architecturally Significant Requirements (ASRs) are requirements that have a direct and substantial impact on the architectural design of the system. For the QR Code-Based In-Table Food Ordering System, the proposed ASRs are derived from the key quality attributes identified in Section 2.2 and serve as the primary drivers for selecting and structuring the modular monolithic and layered design architecture.

ASR-01: Maintainability and Modifiability

- Type: Quality Attribute
- Description: The system must support frequent updates to menu items, localized pricing adjustments, and seamless integration of third-party payment gateways (e.g., MoMo, VNPay) with minimal impact on core transactional workflows..
- Architectural Impact: This requirement drives the adoption of a layered architecture layout, where responsibilities are clearly separated into presentation, service, domain, and data access layers. Such separation allows API integration changes and payment gateway webhooks to be localized within specific domain layers, improving maintainability and reducing coupling between the catalog and the checkout components.

ASR-02: Real-Time Order Synchronization & Notification

- Type: Functional / Quality Attribute
- Description: The system must support immediate, real-time transmission of tableside order placements and urgent assistance alerts from client browsers directly to the merchant staff terminal dashboard without page reloads.
- Architectural Impact: This requirement influences the communication protocol layer, driving the implementation of an asynchronous event-driven model (such as WebSockets or Server-Sent Events) alongside a stateless backend. Centralized event handling within the monolithic architecture guarantees instant alert propagation to active staff sessions and ensures consistent session state management across multiple physical tables.

ASR-03: Security and Data Validation Integrity

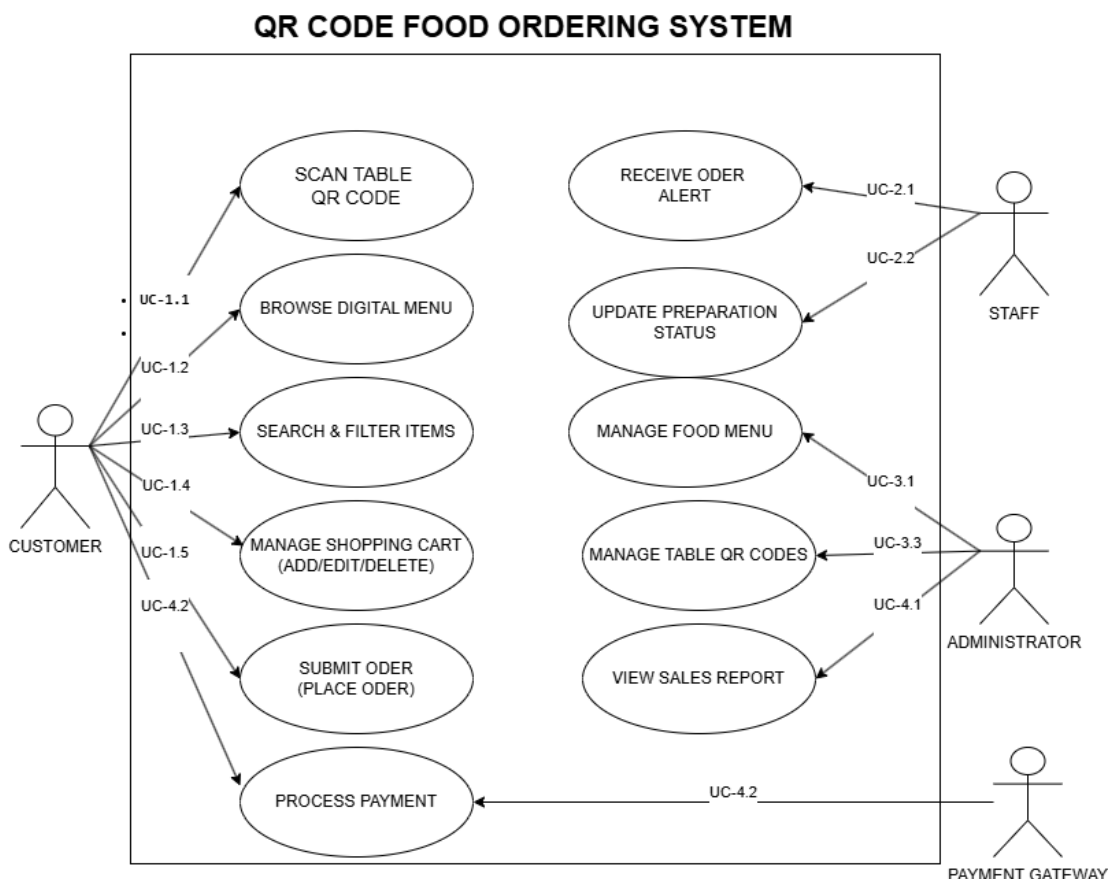
- Type: Quality Attribute
- Description: The system must strictly secure all transactional calculations,

preventing unauthorized client-side parameters or malicious modifications to raw item prices during checkout operations. must maintain strong data consistency and integrity

- This requirement directly dictates the reinforcement of isolated business validation logic within the core service layer. Every inbound table submission payload triggers mandatory server-side cross-referencing against the trusted database pricing engine before a record transitions to active preparation queues, ensuring absolute system integrity.

2.4. Use Case Modeling

The use case diagram illustrates the functional scope of the Curriculum Management



The use case diagram establishes the scope of the QR Code-Based In-Table Food Ordering System by presenting the interactions between different user roles and the core system functionalities. The system boundary represents the tableside ordering and restaurant management platform, while the actors placed outside the boundary indicate the primary stakeholders and external dependencies interacting with the system.

The diagram identifies multiple user roles, including customers, service staff, administrators, and the external payment gateway. Each actor interacts with a specific set

of use cases that reflect their responsibilities within the operational flow. Administrative users are primarily responsible for menu architecture design, table layout coordination, and global configuration-related tasks. These use cases support the foundational setup of the system and ensure that the physical restaurant layout, tax standards, and core pricing parameters are maintained consistently.

The left and central parts of the diagram focus on active customer ordering and operational management, which represent the core business capabilities of the system. Use cases such as digital menu browsing, item searching/filtering, real-time shopping cart manipulation, and immediate table order placement are logically grouped together to emphasize their close relationship within the same transactional domain. The diagram also highlights that staff tasks—such as receiving instant order alerts and updating kitchen preparation states—are managed through controlled, real-time communication processes, ensuring that tableside service updates follow synchronized operational flows..

In addition, the diagram includes use cases related to financial fee unit configuration and bill calculation information display. These use cases are logically separated to indicate that while final checkout processing and tax calculations depend heavily on live order session data, they follow strict architectural isolation to protect data integrity. This separation reflects a heavily secured and validation-oriented interaction pattern, ensuring that pricing modifications do not corrupt processing table bills.

Overall, the use case diagram is intentionally designed at a high level to capture only architecturally significant use cases. Supporting or trivial use cases, such as initial user authentication, network re-connection, or standard profile management, are omitted to maintain focus on functionality that directly influences critical architectural decisions. This modeling approach helps clarify system scope, actor responsibilities, and the relationship between core hospitality business processes without overwhelming the architecture analysis with excessive detail..

3. Architectural Design & Implementation

3.1. Architectural Pattern

The QR Code-Based In-Table Food Ordering System adopts a Modular Monolithic Architecture as its primary architectural pattern. This architectural style combines the simplicity of a monolithic deployment with the structural benefits of modularization, making it well-suited for the project scope, high-concurrency tableside operational demands, and rapid service deployment context.

In a modular monolith, the system is deployed as a single application unit but internally organized into well-defined, highly cohesive modules. Each module encapsulates a specific hospitality business capability (such as menu presentation, table cart management, or financial transaction calculation) and exposes its functionality through clearly defined internal interfaces. Direct access to internal implementation details of other modules is strictly restricted, which helps reduce coupling, isolate runtime errors, and improve long-term maintainability.

Rationale for Choosing a Modular Monolithic Architecture

The decision to adopt a modular monolithic architecture is driven by the following critical technical factors:

Clear Separation of Business Domains: The restaurant system naturally decomposes into distinct, isolated business domains such as digital menu cataloging, live shopping cart operations, table tracking infrastructure, and secure checkout processing. Modularization allows each operational domain to be implemented, tested, and evolved independently within the same application shell.

Maintainability and Modifiability: By enforcing strict module boundaries, changes to one functional area—such as adjusting financial tax structures, modifying third-party payment gateway integration webhooks, or updating layout algorithms for table QR codes—have minimal to zero impact on other operational modules. This structural isolation directly supports the ASRs related to high maintainability and low regression risk.

Strong Data Consistency: Because the food ordering system operates within a single physical deployment unit and a centralized database, transactional consistency across interrelated modules can be easily maintained via standard ACID transactions. This is particularly important for real-time checkout operations where the bill amounts, applied promotional vouchers, and final database sales records must perfectly align without the risk of distributed network partition failures.

Reduced Operational Complexity: Compared to a complex microservices setup, a modular monolith completely avoids the immense overhead of distributed network communication, remote service orchestration, API gateway routing delays, and complex multi-container deployment pipelines. This simplicity minimizes system processing latency—which is crucial for tableside browser responses—and fits perfectly within the capabilities of a small development team operating under a strict project timeline.

Future Evolution Readiness: The clean modular structure provides a solid architectural foundation for potential future scaling. If specific components, such as the real-time order alert or payment processing modules, encounter massive user traffic spikes during peak hours, they can later be extracted into independent, highly scalable services with minimal code refactoring.

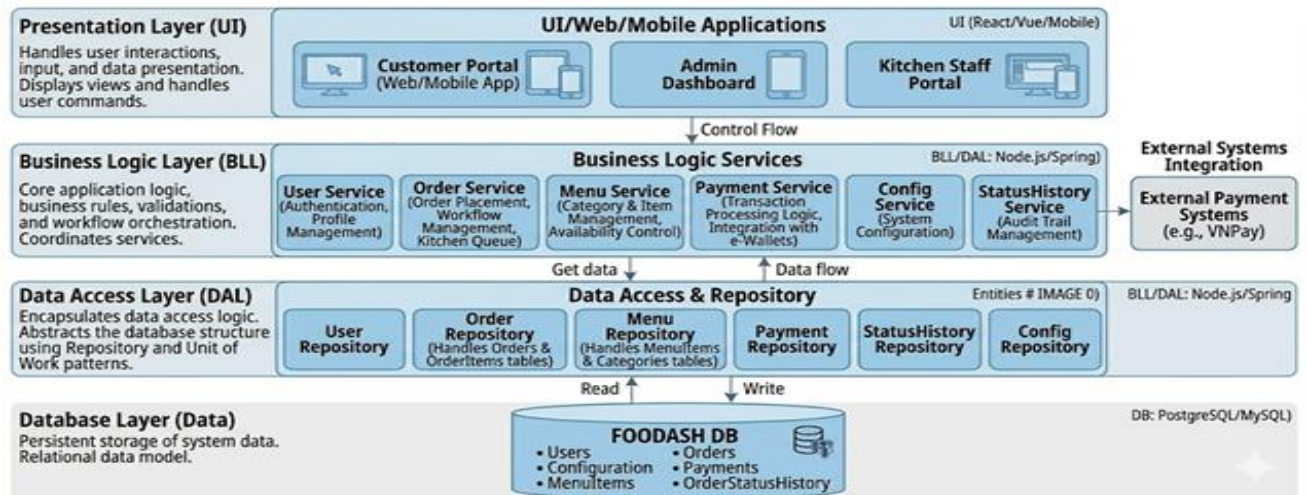
Modular Structure Overview

The system is organized into the following core modules:

- **Menu & Catalog Management Module:** Responsible for structuring food and beverage categories, individual item configurations, pricing adjustments, active promotional combos, and versioning across different seasonal themes or kitchen operational shifts. This module encapsulates all business rules related to menu creation, availability updates, and catalog publication.
- **Table Tracking & QR Management Module:** Handles physical restaurant layout configurations, individual table status tracing, and the mapping or generation of unique tokenized tableside QR code assets. It ensures that incoming order transactions are securely bound to specific physical locations within the premises.
- **Order & Assistance Real-Time Workflow Module:** Manages the active state lifecycle of tableside orders (from placement, kitchen dispatch notification, to tableside delivery) and handles the propagation of urgent customer assistance requests directly to the service staff terminal interface via active real-time communication protocols.
- **Billing & Financial Calculation Module:** Governs global configuration metrics including tax frameworks (VAT), service charge parameters, and localized voucher deductions. It processes multi-item cart computational flows to display accurate real-time bill calculations for checkout validation.

Each module communicates with others through well-defined interfaces at the application/service layer, while direct access to internal domain models or data repositories of another module is avoided.

Layered Architecture Structure



3.2. System Context Diagram (C4 Level)

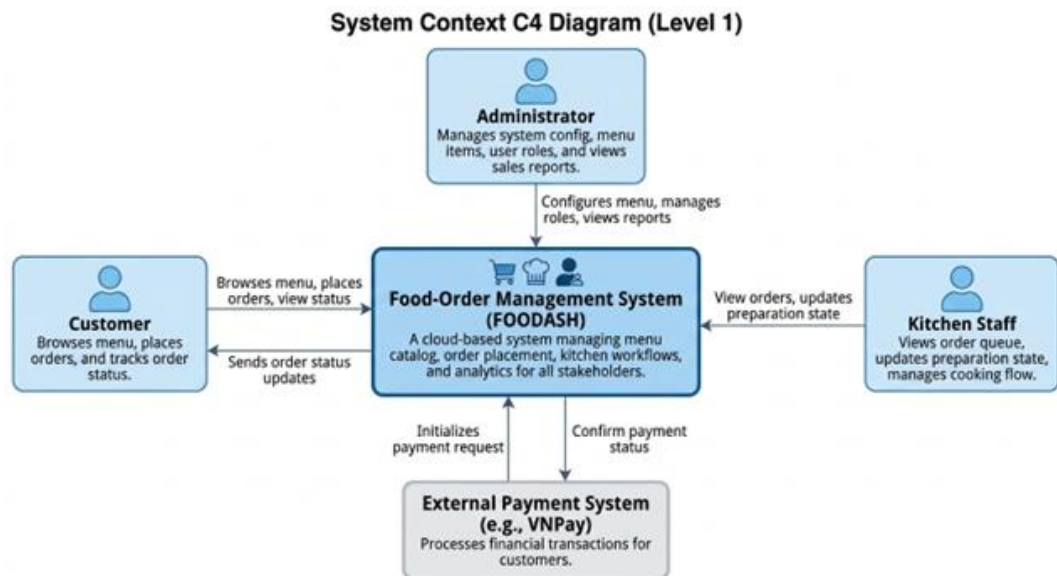
Level 1 : System Context Diagram

At Level 1 - System Context Diagram, the focus is on defining the overall scope of the system and identifying the external actors and systems that interact with it, without going into internal technical implementation details.

FOOD ORDERING SYSTEM is the core system responsible for managing the menu catalog, tableside order placement, kitchen workflows, and operational analytics for all restaurant stakeholders.

- **Customer:** Accesses the digital menu catalog via tableside QR codes, browses available items, modifies shopping carts, submits food orders, and tracks real-time order preparation status.
- **Kitchen Staff:** Utilizes the dashboard interface to view active incoming order queues, update item preparation states, and seamlessly manage internal cooking and dispatch workflows.

- **Administrator:** Manages core system configurations, handles menu item entries and status toggles, assigns user roles or service permissions, and reviews historical sales reports.
- **External Payment System (e.g., VNPay):** Processes secure client-side financial transactions. FOODASH initializes payment requests directed to this external gateway and subsequently receives secure webhooks or payment notifications to confirm transaction statuses..



Level 2 : Container Diagram

The diagram illustrates the container-level architecture of the FOOD ORDERING SYSTEM and shows how users interact with the system and how internal containers communicate with each other.

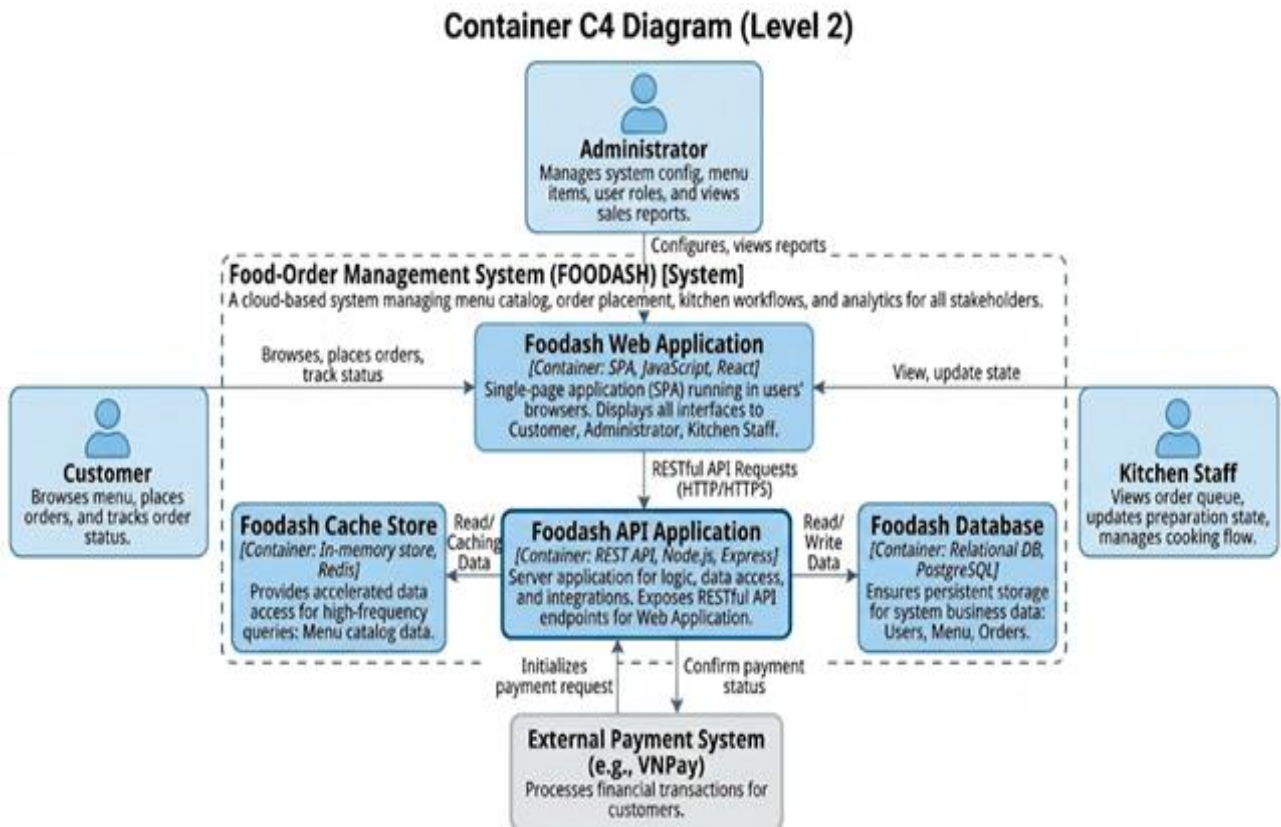
As detailed in level 2 users such as Customers, Administrators, and Kitchen Staff access the system through a web browser using secure protocols. The web browser loads the user interface of the system and forwards user actions to the the FOOD ORDERING SYSTEM

FOOD ORDERING SYSTEM is a Single-Page Application (SPA) built with React and JavaScript running directly in users' browsers. It is responsible for presenting all user interfaces and handling user interactions across all roles (browsing menus and placing orders for Customers, configuring layouts for Administrators, and updating states for Kitchen Staff). It sends RESTful API requests over HTTP/HTTPS in JSON format to the FOOD ORDERING SYSTEM and receives responses to display back to the users.

The FOOD ORDERING SYSTEM acts as the central backend component of the system, implemented as a Node.js and Express REST API server application. It processes the core business logic, data access mechanisms, and external integrations. For persistent storage, the API application reads from and writes data to the FOOD ORDERING SYSTEM Database, a relational PostgreSQL database container where core business data—including Users, Menu items, and Orders—is securely stored.

To fulfill the requirements for high performance and concurrency during peak hours, the API application interfaces directly with the FOOD ORDERING SYSTEM Cache Store. This container utilizes an in-memory Redis database to provide accelerated data access and caching for high-frequency queries, specifically targeting the real-time menu catalog data to prevent processing bottlenecks.

Finally, FOOD ORDERING SYSTEM API Application communicates with the External Payment System (e.g., VNPay) to handle financial checkout calculations. It initializes secure payment requests to the gateway and handles incoming webhooks to accurately confirm payment status updates back to the core infrastructure..



Level 3 Component Diagram:

At Level 3 - Component Diagram, the diagram focuses on explaining how the internal architectural components inside the FOOD ORDERING SYSTEM API Application container decompose and collaborate with each other to implement core system capabilities. User requests originate from the frontend layer, where the FOOD ORDERING SYSTEM Web Application sends HTTP REST requests over secure channels directly to the application endpoints.

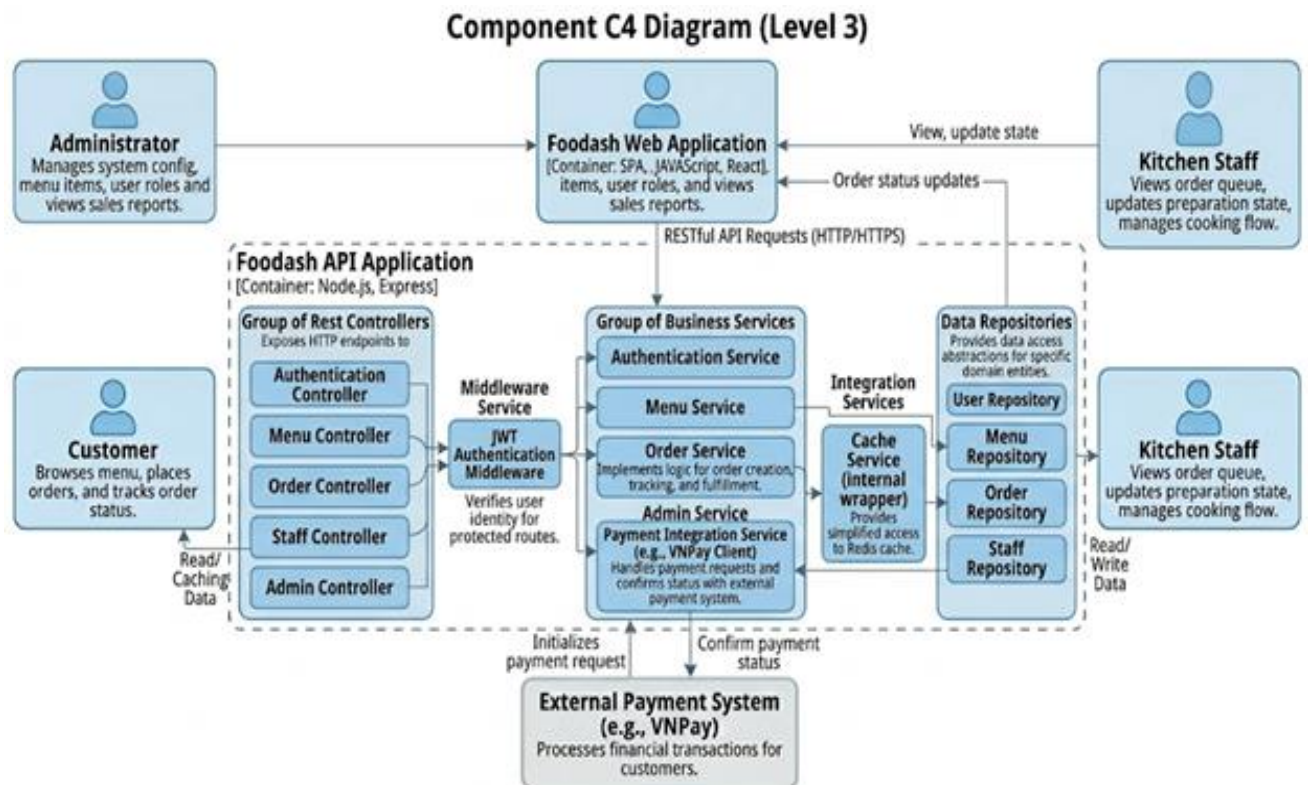
As illustrated in level 3.jpg, the internal architecture of the FOOD ORDERING SYSTEM is organized into three decoupled layers:

Group of REST Controllers: This presentation layer component exposes specific HTTP endpoints to the client browser applications. It comprises the *Authentication Controller*, *Menu Controller*, *Order Controller*, *Staff Controller*, and *Admin Controller*, which intercept inbound requests and delegate payload parameters to the underlying business components after passing through the *JWT Authentication Middleware* for route protection.

These domain events are enqueued to RabbitMQ which provides asynchronous messaging infrastructure. External systems such as the Notification Service consume these events to handle notification related processing. This component level design ensures clear separation of concerns improves maintainability and supports reliable event driven communication within the system

Group of Business Services: Serving as the brain of the container, this core domain layer encapsulates all transactional validation policies. It consists of the *Authentication Service*, *Menu Service*, *Admin Service*, *Order Service* (which handles the logic for tableside order creation, processing lifecycle tracking, and fulfillment), and the *Payment Integration Service* (configured as an external VNPay Client to execute secure checkout routines). These services coordinate data operations via an internal *Cache Service wrapper* linked directly to the external Redis engine.

- **Data Repositories:** Operating as an abstraction barrier over raw database connections, this data access layer contains specialized entity gateways including the *User Repository*, *Menu Repository*, *Order Repository*, and *Staff Repository*. These repositories handle low-level read and write database queries executing against the central PostgreSQL database instance.



Level 4 Code / Class Level:

3.3. Technical Stack and Data Model

The selection of the technology stack for the FOOD ORDERING SYSTEM is driven by the core Architecturally Significant Requirements (ASRs), focusing on low latency for tableside operations, high concurrency handling during peak hours, and strict data security.

Technical Stack

- **Rationale:** Built on Google Chrome's V8 engine, Node.js utilizes an event-driven, non-blocking I/O model. This lightweight nature allows the backend application container to process hundreds of concurrent request-response payloads from table browsers simultaneously with minimal server resource consumption.

Presentation Layer: A web-based single-page user interface (SPA) is used to interact with the system across different device roles (Customers, Staff, and Administrators). The presentation layer communicates with the application layer through well-defined controllers and secure API endpoints, ensuring absolute separation between dynamic user interaction and core restaurant business logic.

- **Database Management System:** A relational database management system

(RDBMS), specifically PostgreSQL, is used to store persistent operational data. The choice of a relational database supports strong transactional consistency (ACID compliance) and structured data modeling, which are essential for managing real-time transactional information such as table order logs, processing status, and financial billing details.

- **Persistence Technology:** An object-relational mapping (ORM) frameworks or structured data access abstraction layers are utilized to manage interactions between the backend application services and the database engine. This approach simplifies data access logic, protects against SQL injection vulnerabilities, and significantly improves system maintainability.
- **Security and Authorization:** Role-based access control (RBAC) combined with JSON Web Token (JWT) verification is implemented at the application layer to ensure that only authenticated and authorized users can perform sensitive administrative operations, such as modifying menu pricing configurations, updating table mapping layouts, or viewing financial sales data.

Data Model Overview

The data model is designed to reflect the core food-service and transactional concepts of the system while supporting menu versioning, table tracking, and absolute financial data integrity. The main entities include:

Program: Represents a training program offered by the institution. A program may have multiple curriculum versions over time.

- **Menu (Catalog):** Represents the overall food and beverage catalog offered by the restaurant. A menu layout may have multiple seasonal versions or active shift adjustments over time.
- **MenuVersion:** Represents a specific approved version of the food menu. Each version defines a fixed set of items, active modifier rules, and reference base prices that apply to the current service shift. Once published and active, historical menu configurations tied to finalized orders are treated as immutable to ensure absolute financial consistency.
- **MenuItem:** Represents an individual food or drink offering, including specific attributes such as item code, item name, price, stock availability status, and cooking preparation metrics.
- **OrderLineItem (OrderDetail):** Defines the transactional relationship between an active table order and its associated menu items, including additional attributes such as ordered quantity, specific customer notes, and the historical unit price snapshot captured at order time.

- **User:** Represents an internal system user, including administrators and service staff
- **Table:** Represents a physical seating asset within the restaurant layout, mapped securely to a unique tokenized QR code identifier for accurate tableside order routing.
- **Role:** Defines system user roles and localized data permissions used to strictly enforce role-based access control across administrative dashboards.

Modular Data Ownership

In line with the modular monolithic architecture, each module within the FOOD ORDERING SYSTEM owns and manages its corresponding data and domain logic. For example, the Menu & Catalog Management Module is strictly responsible for menu items, base pricing adjustments, and layout version entities, while the Table Tracking & QR Management Module independently manages table configurations and tokenized QR asset data. Although all modules share the same physical database container, logical ownership boundaries are rigorously maintained at the application/service level to reduce data coupling and improve architectural clarity.

Data Consistency and Integrity

Strong data consistency is ensured through centralized transaction management within the monolithic application runtime. Business rules—such as preventing any modifications to published menu versions currently active at tables, or disallowing the deletion of menu items that are bound to active, uncompleted customer order rows—are strictly enforced at the service and domain layers. This approach simplifies consistency management and directly aligns with the system’s primary Architecturally Significant Requirements (ASRs).

3.4. Implementation Overview

This section provides an overview of how the Food-Order Management System is implemented based on the proposed modular monolithic architecture. The implementation focuses on enforcing clear module boundaries, centralized tableside business logic, and consistent data handling within a single deployable application container.

Overall Application Structure

The system is implemented as a single monolithic application, internally organized into multiple independent, highly cohesive modules. Each module represents a specific restaurant business capability (e.g., Menu Catalog, Live Ordering, Table Tracking, and

Checkout Billing) and contains its own presentation interfaces (controllers), application services, domain models, and data access components.

Although all modules are packaged and deployed together, strict structural coding conventions are applied to prevent tight coupling and to preserve clean modularity. Interactions between different operational modules occur only through explicitly exposed service interfaces or contract DTOs (Data Transfer Objects), rather than direct cross-module modifications to internal data repositories or implementation details.

Module-Level Implementation

As visualized in the Component C4 Diagram (level 3), each internal module follows a consistent, multi-layered internal structure:

- **Controller / Interface Layer:** Handles incoming asynchronous RESTful HTTP/HTTPS requests from the client-side Single-Page Application (React Frontend). This layer is responsible for structural payload validation, route token verification, request mapping, and invoking the appropriate backend application services.
- **Application / Service Layer:** Implements granular restaurant use case logic (such as calculating multi-item cart subtotals or triggering staff notifications) and coordinates seamless interactions between core domain objects and data repositories. This layer acts as the primary integration point between modules, ensuring cross-module communication follows defined interface contracts.
- **Domain Layer:** Encapsulates the core business rules and entities such as Menu items, Active Orders, Table tokens, and Tax percentages. Domain logic strictly enforces operational invariants, such as the absolute immutability of pricing models bound to active, processing table checkouts.
- **Data Access Layer (Repositories):** Manages persistence operations using specific repository abstractions (e.g., *Order Repository*, *Menu Repository*). This layer completely isolates database-specific raw SQL logic and ensures absolute consistency in data retrieval and storage operations against the PostgreSQL instance.

Inter-Module Communication

Inter-module communication is implemented through explicit service interfaces defined strictly at the core application layer. For example, when a customer places an order, the Order & Assistance Real-Time Workflow Module queries specific item availability and current pricing configurations through the Menu & Catalog Management Module's exposed service interface, rather than querying the menu data tables directly.

This decoupled approach reduces dependency chains and fully supports the future evolution of the platform, including the potential extraction of specific heavy modules (like the real-time notification loop) into standalone microservices if required by traffic growth.

Transaction Management

Transaction boundaries are managed centrally within the monolithic backend engine. Complex operations that involve multiple domain entities or different modules—such as processing a checkout session, which simultaneously transitions an order status to "Paid", updates the corresponding physical table state back to "Available", and logs an immutable gateway transaction—are executed safely within a single database transactional context to ensure absolute atomicity. This design simplifies application error handling and aligns with the strong transactional consistency requirements of the platform.

Security Enforcement

Security, identity validation, and authorization checks are implemented consistently across both the service and controller layers. Role-Based Access Control (RBAC) backed by JWT verification ensures that only authorized administrators can perform sensitive administrative mutations, such as updating global tax frameworks, deleting table layouts, or auditing sales logs, while kitchen staff sessions are restricted to order queue status modifications. Centralized security middleware drastically reduces code duplication and improves system maintainability.

Deployment Considerations

The entire the FOOD ORDERING SYSTEM backend application is compiled and deployed as a single runtime unit, significantly simplifying standard server configuration, infrastructure monitoring, environment orchestration, and continuous deployment pipelines. This deployment model is exceptionally well-suited for fast hospitality business setups and fits perfectly within the project scope, while the clean internal modular architecture guarantees that the codebase remains maintainable, testable, and robust against regression bugs as the system scales.

4. Testing & Verification

5. Conclusion & Reflection

This project presented the design and implementation of a QR Code-Based In-Table Food Ordering System based on a modular monolithic architecture. The primary objective was not only to build a functional, consumer-ready e-menu platform but also to apply software architecture principles in a structured and technically justifiable manner.

Starting from core restaurant business requirements, the project identified key quality attributes and architecturally significant requirements (ASRs) that guided critical system decisions. The selected modular monolithic architecture successfully balances development simplicity, strict localized maintainability, and immediate tableside data consistency, making it exceptionally suited for the scope of the project and the fast-paced hospitality environment. Through systematic testing and architectural validation, the system demonstrated that it fully meets both functional tableside operations and non-functional engineering requirements, particularly in terms of low-latency performance, transaction security, and concurrent reliability.

Overall, the project successfully achieved its learning objectives by translating theoretical architectural concepts into a practical, robust system design and implementation.

5.1. Lessons Learned

One of the most important lessons learned from this project is that software architecture must be fundamentally driven by concrete requirements rather than fleeting technology trends. By carefully analyzing functional ordering workflows and critical quality attributes, the team was able to mathematically and logically justify the adoption of a clean modular monolithic architecture instead of choosing more complex, distributed

microservices approaches. This reinforced the engineering understanding that an architecture is considered effective only when it perfectly aligns with the actual operational needs and infrastructure constraints of the target system.

The project also highlighted the immense value of strict modularity within a monolithic runtime deployment shell. Although the FOOD ORDERING SYSTEM application is packaged and deployed as a single unit, enforcing rigid module boundaries at the service layer significantly improved long-term maintainability and minimized dangerous regression coupling between completely different parts of the system—such as preventing catalog price updates from corrupting live table checkout calculations. This experience effectively demonstrated that monolithic architectures, when engineered with proper domain boundaries, can easily achieve high levels of structural clarity, isolation, and extensibility.

Another key lesson was the paramount importance of structured, early architectural modeling. Utilizing engineering techniques such as high-level use case modeling, hierarchical C4 diagrams (Context, Container, and Component levels), and ASR analysis helped clarify the application's true scope and justified technical choices before a single line of implementation code began. These models served as an unyielding common reference point throughout the development process, reducing misunderstandings and guiding consistent code patterns. Additionally, systematic performance testing played a crucial role in validating architectural assumptions, particularly regarding low-latency response times and data precision under simulated concurrent user tableside access during peak dining hours.

5.2. Future Improvements

Although the current system satisfies the project requirements, several improvements can be considered for future development. From a scalability perspective, performance can be further enhanced through advanced database optimization, query caching mechanisms, or infrastructure resource tuning as tableside user demand increases. If the system grows significantly in size or operational complexity during peak restaurant hours, the existing modular structure provides a solid foundation for gradually extracting selected modules, such as the real-time order synchronization core, into independent microservices.

Security can also be strengthened by introducing advanced mechanisms such as

automated transactional audit logging, finer-grained role authorization policies, and enhanced device authentication methods. These improvements would further protect sensitive financial parameters, block client-side data mutations, and support stricter restaurant governance requirements. In addition, the user experience could be improved by refining the interactive e-menu interface, enhancing smart item search and category navigation features, and providing better multi-device visualization of the order lifecycle and kitchen preparation queues for different restaurant stakeholders.

Finally, future work may include the adoption of comprehensive automated testing packages and real-time application monitoring tools to improve long-term platform reliability and backend maintainability. Continuous testing loops and centralized system monitoring dashboards would allow early detection of connection dropouts or transactional failures, supporting more efficient and safe system evolution..